Neural Computing and Applications (2020) 32:12499–12514
https://doi.org/10.1007/s00521-020-04708-x

ORIGINAL ARTICLE

An efficient XGBoost–DNN-based classification model for network intrusion detection system

Preethi Devan[1] · Neelu Khare[1]

Received: 29 October 2018 / Accepted: 7 January 2020 / Published online: 19 January 2020
© Springer-Verlag London Ltd., part of Springer Nature 2020

Abstract

There is a steep rise in the trend of the utility of Internet technology day by day. This tremendous increase ushers in a massive amount of data generated and handled. For apparent reasons, undivided attention is due for ensuring network security. An intrusion detection system plays a vital role in the field of the stated security. The proposed XGBoost–DNN model utilizes XGBoost technique for feature selection followed by deep neural network (DNN) for classification of network intrusion. The XGBoost–DNN model has three steps: normalization, feature selection, and classification. Adam optimizer is used for learning rate optimization during DNN training, and softmax classifier is applied for classification of network intrusions. The experiments were duly conducted on the benchmark NSL-KDD dataset and implemented using Tensor flow and python. The proposed model is validated using cross-validation and compared with existing shallow machine learning algorithms like logistic regression, SVM, and naive Bayes. The classification evaluation metrics such as accuracy, precision, recall, and F1-score are calculated and compared with the existing shallow methods. The proposed method outperformed over the existing shallow methods used for the dataset.

Keywords IDS · Deep neural network (DNN) · XGBoost · NSL-KDD

1 Introduction

The development of information technology has led to an increase in attackers. Early attacks were performed mainly for fun and curiosity about the technology. However, in recent years, such attacks are explosively increased and cause substantial economic loss to the organizations based on computer networks. Efficient and effective design and implementation strategies to beat cyber-crimes have become necessary. The major challenge in network security is to implement efficient intrusion detection system (IDS). In this paper, we focus on the three essential limitations which make difficulties in the design and implementation of IDS. They are a high volume of data, in-depth network monitoring, and diversity of data and different types of protocols [1]. The volume of data is drastically continuing to increase day by day. As there is an increase in the levels of network connectivity, extensive usage of the Internet of Things, online education, e-commerce, and adoption of cloud-based services have led to the increase in the volume of data. The in-depth network monitoring is also required as the attackers are changing their attack patterns frequently.

There are many techniques to mitigate such attacks which include intrusion detection systems, statistics-based systems, anomaly-based, and signature-based systems. Intelligent systems include machine learning algorithms, artificial neural network-based, fuzzy logic-based, and genetic algorithm-based detection systems. Network intrusion detection systems are used to identify a set of malicious activities that threaten the integrity, confidentiality, and availability of a network resource. Intrusion detection is of two types: misuse (signature)-based detection and anomaly-based detection. Misuse-based detection searches for specific patterns and user behavior that match well-known intrusion scenarios. Anomaly-based detection develops models of normal network behavior, and new intrusions are detected by evaluating significant deviations from normal behavior [2]. However,

✉ Neelu Khare
neelu.khare@vit.ac.in

Preethi Devan
preethi.d2014phd0201@vit.ac.in

[1] School of Information Technology and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

signature-based detection is not capable of detecting new and unfamiliar intrusions, even if they built as minimum variants of already known attacks.

On the other hand, the advantage of anomaly-based detection techniques is their potential to detect previously unseen intrusion events. However, the rate of false positives in anomaly-based systems is usually higher than in signature-based ones [3]. The majority of the solutions existing today focus on the signature-based techniques. These techniques suffer from high false error rates and an increase in computing cost. Moreover, existing IDS are designed with strict rules, and they are easily prone to false-positive and false-negative alarms. These high false detection rates make it challenging to install IDS in practical large-scale systems. In recent years, the research in the field of network IDS has focussed on the shallow machine learning techniques such as *k*-nearest neighbor [4], SVM [5], naive Bayes, random forest [5, 6], decision trees, and artificial neural networks [7]. The application of shallow learning algorithms has offered improved detection accuracy. On the other hand, these methods need domain expert knowledge to identify useful patterns from the data; the computing cost is expensive, and it is also prone to errors.

The recent research on deep learning [8], which is receiving attention, such as speech recognition and image processing is applied in this paper. Deep learning is the subset of machine learning algorithms which can overcome some of the limitations of shallow learning algorithms. Deep learning focuses on the faster detection of network anomalies as well as more in-depth network data analysis. Its layer-wise feature learning shows better results when compared with the performance of shallow learning techniques. On DNN, there is an increase in the number of features which may lead to overfitting of the training data. To avoid overfitting, we use eXtreme Gradient Boosting (XGBoost) [9] feature importance score for reducing the number of features. XGBoost algorithm is used in top challenges hosted by Kaggle competitions that motivated us to propose IDS based on XGBoost–DNN-based network intrusion detection.

The main contributions of this paper are listed as follows:

- Design and implementation of an efficient XGBoost–DNN classifier model for classification of network intrusions.
- The benchmark NSL-KDD dataset used for the experimentation and testing of the proposed model.
- The performance of the proposed XGBoost–DNN is verified by comparison with the three existing shallow learning algorithms.

The rest of this paper is organized into the following sections: Sect. 2 examines the literature review. Section 3 presents the proposed approach explaining about the XGBoost–DNN-based network intrusion detection. Section 4 shows the experimental results and a comparison with the existing shallow machine learning techniques.

2 Literature review

This section discusses the literature works based on the machine learning approaches for the network intrusion detection system. There are very few publicly available benchmark datasets available for the research work based on network intrusion detection system. Some of them include: KDD CUP' 99, NSL-KDD dataset, CAIDA 2007, and UNSW-NB15. We have used NSL-KDD benchmark dataset for the experimentation of our proposed work. Shone et al. [1] present a nonsymmetric deep autoencoder (NDAE) for unsupervised feature learning in IDS. They have constructed classification models using stacked NDAE's. The experiments were performed on the standard dataset NSL-KDD using GPU-based Tensor flow. The authors have achieved a promising classification accuracy for both 5-class and 13-class classification models when compared with other existing techniques.

The work has experimented only on the synthetic data. Niyaz et al. [10] proposed self-taught learning (STL) which is a deep learning model for the intrusion detection system. The experiments were conducted using benchmark NSL-KDD. The authors claimed that they had achieved consistent levels of classification accuracy in both binary and 5-class classification. Kim et al. [11] focused on a deep neural network model for intrusion detection. The KDD CUP99 dataset is applied for implementation on a GPU-based Tensor flow. All the 41 features in the dataset and the proposed model lack feature selection techniques. They have built a deep neural network with 100 hidden units, rectified linear units (ReLU) as activation function and Adam optimizer. The authors claimed 99% accuracy and concluded that they need time-series data analysis techniques such as long short-term memory (LSTM) and recurrent neural networks (RNN) models for their future work. Paul et al. [12] presented a CFS-DNN-based intrusion detection system, where correlation-based feature selection (CFS) is used for feature selection. They have used benchmark NSL-KDD for implementation. Adam optimizer is applied to optimize the learning rate during the training process.

Zhang et al. [13] worked on deep learning classifier for software-defined networks (SDN). They have experimented on Moore dataset using WEKA and MATLAB. The authors have achieved good classification accuracy. They have concluded that their proposed work can be extended to unsupervised learning, reducing the training time, and increasing the speed of the network learning capacity. Aljawarneh et al. [14] described a hybrid model

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

for anomaly-based intrusion detection system with a feature selection technique.

The 20% NSL-KDD dataset is used for experimentation. The feature selection is carried out using vote algorithm along with information gain, and the 41 features reduced to 8 features. The hybrid model with classifiers J48, Meta paging, RandomTree, REPTree, AdaBoostM1, decision stump, and naive Bayes is used for classification of intrusions. The model mainly focused on the improved accuracy with high false-positive and low false-negative rate. Moustafa et al. [15] presented an ensemble-based intrusion detection technique to mitigate against protocol-based botnet attacks in IoT. The statistical new flow features are generated, and the essential features are selected using the correlation coefficient. Then, binary classification is carried out using AdaBoost ensemble learning model with a classifiers decision tree, naive Bayes, and artificial neural networks. The experimentation is carried out using statistically generated dataset along with the UNSW-NB15 and the botnet dataset NIMS. The detailed description of our proposed approach and their experimental results are explained in the following sections.

3 Materials and method

3.1 NSL-KDD dataset description

The NSL-KDD dataset is the modified version of the KDD cup 99 dataset [16]. In this dataset, the redundant records are removed and also reduced the reasonable number of records in the training and the testing sets. In the current literature [17–19], the researchers used NSL-KDD dataset as the standard benchmark dataset to demonstrate improvement over IDS for their experimental evaluations. Hence, we have also selected the NSL-KDD dataset for our experimental evaluations. Many researchers have analyzed NSL-KDD by employing various machine learning approaches. A detailed investigation and analysis of machine learning techniques for intrusion detection are carried out in [20]. This paper aims to use the deep neural network based on the binary classification of intrusion detection. This dataset has 41 features shown in Table 1. For this purpose, the NSL-KDD dataset, which has four attack categories shown in Table 2, is converted to single class as abnormal.

3.2 The proposed XGBoost–DNN model for network intrusion detection system

The proposed XGBoost–DNN classifier model as the network intrusion detection system is illustrated in Fig. 1. Firstly, the data are preprocessed using normalization technique. Secondly, the feature selection is made to overcome the problem of the curse of high dimensionality. The unwanted features are eliminated based on feature importance score from XGBoost technique. Thirdly, the DNN classifier is constructed and trained. Lastly, the classification of network intrusion is performed using DNN. The detailed description of each step in the proposed approach is explained in the next sections.

The algorithm as shown below shows the steps involved in the design of the proposed XGBoost–DNN model.

Input: NSL-KDD dataset
Output: NSL-KDD dataset classified with the specific class label either as normal or attack

Algorithm Steps:

Step 1: Apply data cleaning to the input NSL-KDD dataset.
Step 2: Normalization of the NSL-KDD using min–max technique.
Step 3: Apply XGBoost feature importance score for feature selection.
Step 4: Construct the deep neural network classifier with the selected feature set from Step 2.
Step 5: Training the DNN classifier.

Table 1 Features in NSL-KDD dataset

S. no	Feature name	S. no	Feature name
1	Duration	23	Count
2	Protocol_type	24	Srv_count
3	Service	25	Serror_rate
4	Flag	26	Srv_serror_rate
5	Src_bytes	27	Rerror_rate
6	Dst_bytes	28	Srv_rerror_rate
7	Land	29	Same_srv_rate
8	Wrong_fragment	30	Diff_srv_rate
9	Urgent	31	Srv_diff_host_ rate
10	Host	32	Dst_host_count
11	Num_failed _logins	33	Dst_host_srv_ count
12	Logged_in	34	Dst_host_same _srv_rate
13	Num_compromised	35	Dst_host_diff_ srv_rate
14	Root_shell	36	Dst_host_same _src_port_rate
15	Su_attempted	37	Dst_host_srv_ diff_host_rate
16	Num_root	38	Dst_host_serror_rate
17	Num_file_creations	39	Dst_host_srv_serror_rate
18	Num_shells	40	Dst_host_rerror_rate
19	Num_access_files	41	Dst_host_rerror_rate
20	Num_outbound_cmds		
21	Is_hot_login		
22	Is_guest_login		

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 2 Different attack types in NSL-KDD dataset

Attack class	DoS	Probe	R2L	U2R
Attack type	Back	Satan	Guess_Password	Buffer_overflow
	Land	Ipsweep	Ftp_write	Load module Rootkit
	Neptune	Nmap	Imap	Perl
	Pod	Portsweep Mscan	Phf	SQL attack
	Smurf	Saint	Multihop Warezmaster	X term
	Teardrop		Warezclient	Ps
	Apache2		Spy	
	Udp storm		Xlock	
	Process table		Xsnoop	
	Worm		Snmpguess	
			Snmpgetattack	
			Httptunnel	
			Sendmail	
			Named	

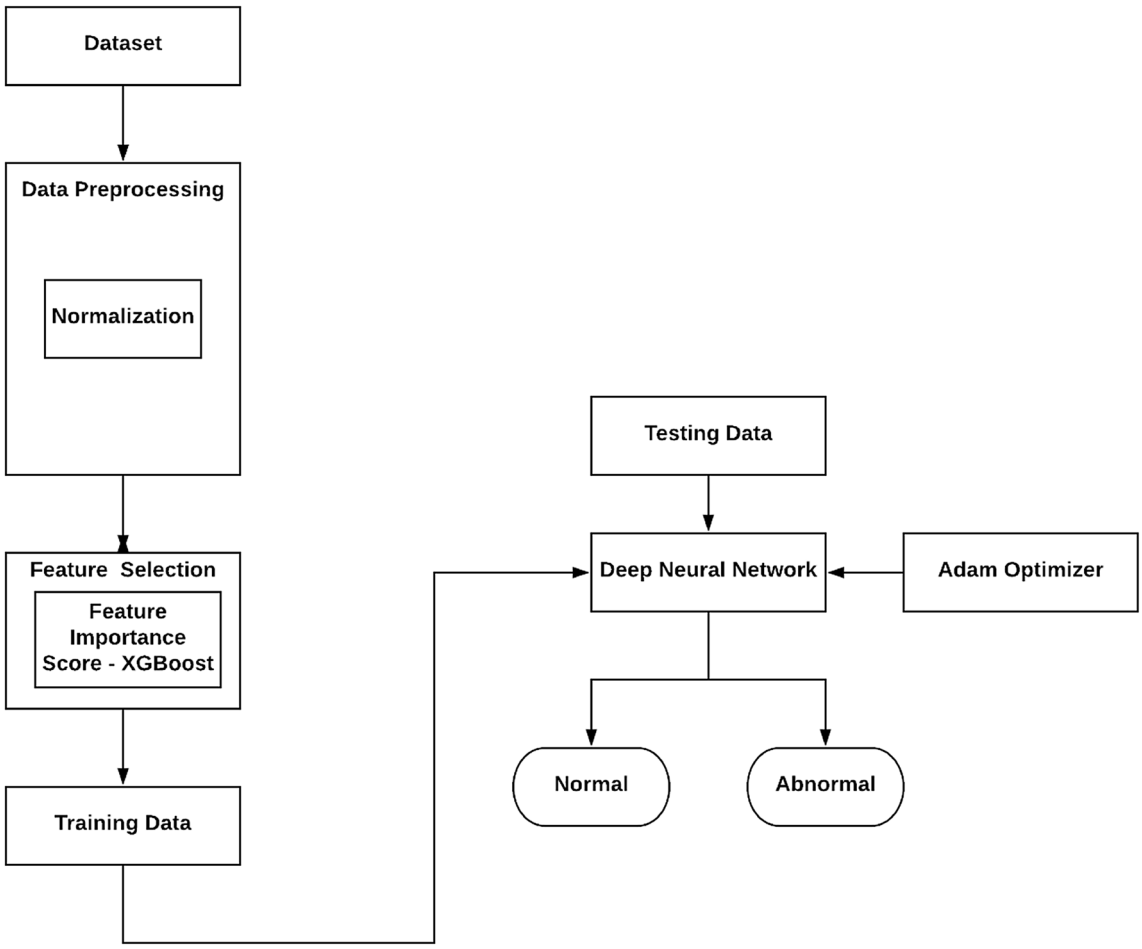

Fig. 1 Proposed XGBoost–DNN classifier model for network intrusion detection

Step 6: Optimize the DNN classifier constructed in steps 4 and 5 using Adam optimizer.
Step 7: Evaluate the proposed XGBoost–DNN classifier model using cross-validation.

3.2.1 Normalization

Data normalization was especially useful for modeling applications where the inputs are generally used on widely

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

different scales. Min–max normalization provides more flexibility in designing neural networks. This normalization technique has the advantage of preserving precisely all relationships in the data, and thereby, it does not introduce any bias. When the min–max is applied, each feature lies within the suitable range of values for the classifier, but the underlying distributions of the corresponding features within the new range of values remain the same [21]. The NSL-KDD dataset has features with both discrete and continuous values. When the discrete and continuous values of the feature are combined, it makes the range of the feature values different. To solve this, we applied min–max normalizations to fit our calculations as in [12] and this resulted in the same range for the entire feature, which helped to train the model best. NSL-KDD dataset has 41 features which have 3 nominal and 38 numeric features. Many machine learning algorithms and even the neural network training process accept inputs only in a specific form. So, the three nominal features (Protocol_type, service, and flag) are converted into numeric values. Then, all the features are normalized using min–max normalization to transform the dataset into the interval range of [0, 1]. This technique maps the dataset D to the range of D' in the specified range of [new_min, new_max] by using the formula,

$$D' = (D - D\text{min})/(D\text{max} - D\text{min}) \times [\text{new_min} - \text{new_max}] + \text{new_min} \quad (1)$$

wherenew_min and new_max represent the transformation range of the dataset D with the values set as new_min = 0 and new_max = 1.

3.2.2 Feature selection

Next, feature selection is applied, which is an essential task in data categorization. Feature selection has advantages such as reducing computational complexity; increasing the performance of the learning algorithm; eliminating redundant information, and improvement in the generalization and understanding of the data. XGBoost is a scalable machine learning model for tree boosting which is described in [9]. The performance of the XGBoost model is widely recognized in several data mining and machine learning challenges. Thus, we employed XGBoost technique in which the feature importance score is applied for feature selection. The XGBoost model is a collection of decision trees.

The prediction is carried out with the collection of decision trees by

$$y_i' = \sum_{m=1}^{m} f_k \quad (2)$$

where m = number of decision trees, f_k = prediction from a decision tree, x_i = feature vector at the ith data point.For training the model, it is necessary to optimize a loss function. The loss function for our binary classification is given as

$$L = -\frac{1}{N}\sum_{i}^{N}((y_i \log p_i) + (1 - y_i)\log(1 - p_i)) \quad (3)$$

Regularization is another important part of the XGBoost model and is given by

$$\Omega = \Upsilon^{\mathrm{T}} + \frac{1}{2}\lambda \sum_{j=1}^{\mathrm{T}} w_{j^2} \quad (4)$$

where T = number of leaves and w_{j^2} = score on the jth leaf.

The objective of the model is expressed as

$$\text{obj} = L + \Omega \quad (5)$$

where L = training loss which determines how well the model is predictive. Ω = regularization function which controls the model complexity and prevents overfitting.

In XGBoost, we use the mean and variance as the order of gradient descents to optimize the objective function. Thus, the objective function for the step is expressed as

$$\text{obj}(t) = \sum_{i=1}^{n}\left[g_i f_t(x_i) + \frac{1}{2}h_i f_t^2(x_i)\right] + \Omega(f_t) \quad (6)$$

3.2.3 Outline of the XGBoost algorithm

1. For n number of times
   (a) Build a tree until it reaches its maximum depth (each node represents a single feature)
      i. Find the best splitting point
      ii. Assign a weight to the two new leaves
   (b) The tree is pruned to delete the nodes with negative gain in bottom-up order.
2. End for

Based on the above steps, the trees in the XGBoost model are constructed. Let the feature set, v_i be $v_1, v_2, \ldots, v_m$ where i = 1, 2, …, m. Then, the feature importance score IS_i is calculated based on the number of times each feature is used to split the training data across all the trees. Thus, the feature importance score set is expressed as

$$\text{IS}_i = \{x | x = w_i v_i\} \quad (7)$$

where w_i denotes the weight of each feature and v_i denotes the feature set. Figure 2 shows NSL-KDD features with the best feature importance scores plotted using XGBoost algorithm.

The XGBoost has been used as an individual classification model in the literature [22–24]. The accuracy of

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

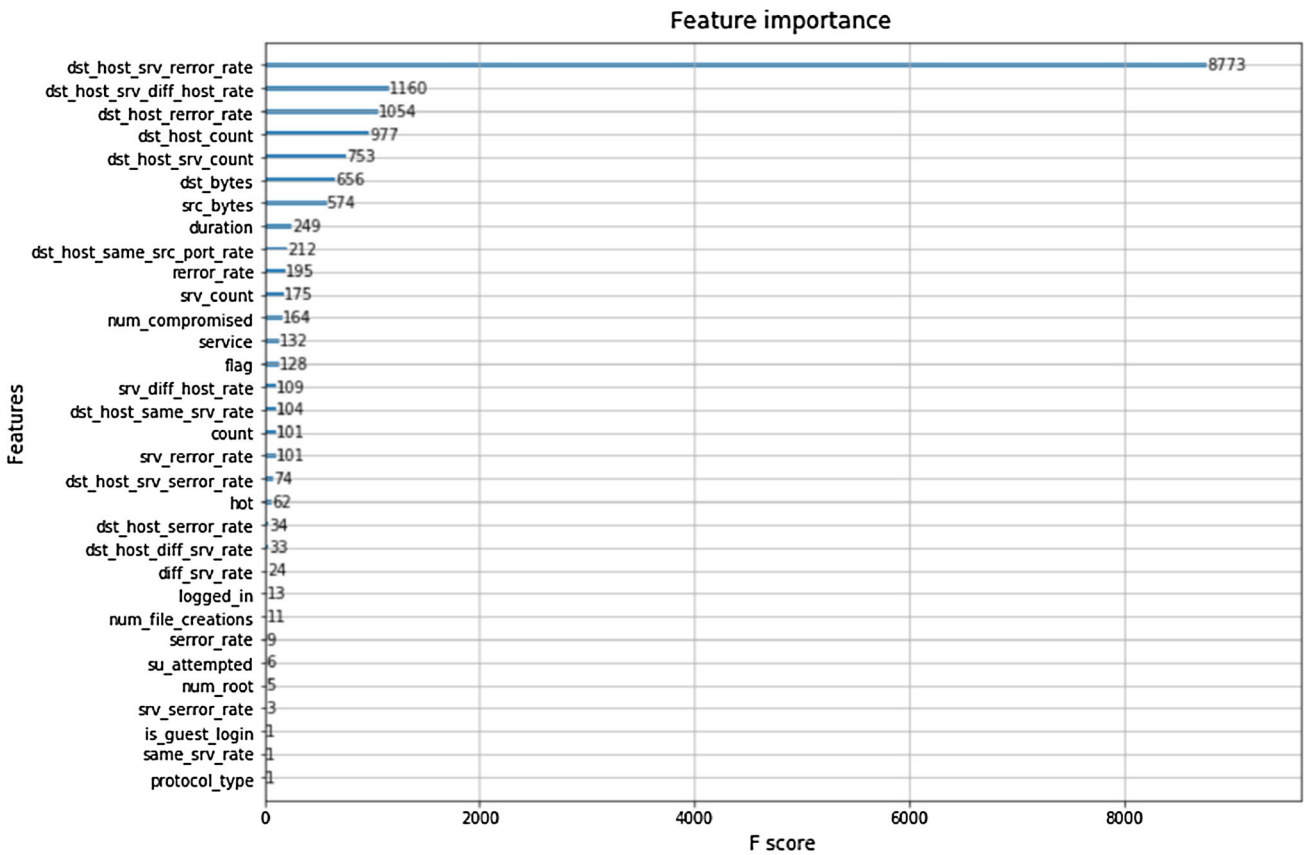

Fig. 2 Feature importance score using XGBoost for NSL-KDD

XGBoost algorithm is tested by extensive experiments with multiple thresholds for selecting features by feature importance score. For each input variable, the feature importance score essentially allows testing each subset of features by importance. The experiment has started using all 41 features and ended with a subset containing the essential features. From Table 3, it can be seen that the performance of the model generally decreases to the number of selected features. There is a trade-off between the number of features and test set accuracy. So, the best possible combination of 21 features (out of 41 features) is selected with 98.91% accuracy. The selected features are then fed into deep neural network for classification of network intrusions.

Table 3 Feature selection using XGBoost feature importance score with multiple thresholds and their respective accuracy

Threshold	Number of features	Accuracy (%)
0.000	41	98.19
0.002	21	98.91
0.004	18	98.03
0.023	17	98.03
0.028	16	97.91
0.032	15	97.03
0.039	14	97.91
0.042	12	98.55
0.046	11	98.42
0.049	10	98.67
0.053	9	98.42
0.054	8	97.33
0.060	7	97.54
0.062	6	98.55
0.072	3	97.33
0.199	1	96.00

3.2.4 Classification

In this step, deep neural network classifier is constructed. Deep neural network (DNN) is a feed-forward neural network, which is a descendant of the classic artificial neural network. Figure 3 illustrates deep neural network architecture. DNN consists of three layers, viz. input, hidden, and output layer. The input layer feeds the pre-processed input data into the network. The number of input neurons is the same as the input features in the dataset to the neural network. Thus, the input layer with D inputs is expressed as

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

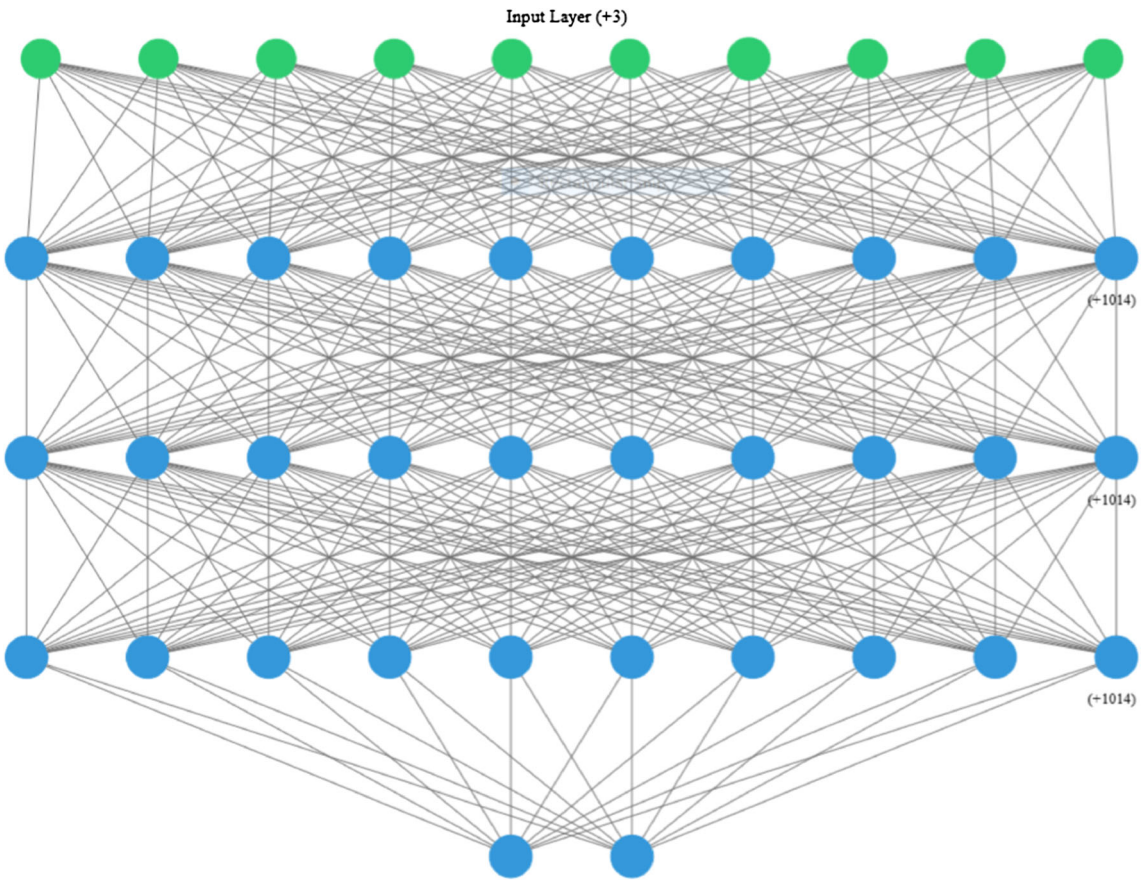

Fig. 3 Deep neural network architecture

$$X = [x_1, x_2, \ldots, x_D] \tag{8}$$

The next layer is the hidden layer, as the deep learning network allows adding more than one hidden layer. The hidden layer maps the input X from the input layer with random weights, $w_{i,}$ and a bias, b_j. Thus, hidden layer inputs are expressed as

$$h_j = \sum_i w_i x_i + b_j \tag{9}$$

where j represents the number of hidden units in the DNN, j = 1, 2, 3, …, k.

Each hidden layer is associated with a nonlinear activation function. ReLU has created a better performance and in accelerating the training process in deep neural networks. The main breakthrough of ReLU was the removal of vanishing and exploding gradient problem. In spite of hard non-linearity and non-differentiability at zero, the rectifier neurons show better results compared to sigmoid and hyperbolic tangent neurons [25]. Thus, creating sparse representations with true zeros provides remarkable results suitable for sparse data. Both sigmoid and tanh activation functions show reasonable detection rates than ReLU for 100 epochs in binary classification. When the experiment is conducted for 500 epochs, the better performance is achieved with ReLU than sigmoid and tanh activation functions. Thus, we decided to apply ReLU activation functions for implementing our proposed model.

The results of the hidden layer are expressed as

$$h = f(h_j) \tag{10}$$

where

$$f(h_j) = ReLU(h_j) \tag{11}$$

The output layer processes the inputs from the hidden layer to the output layer's activation function and produces the outputs of the DNN. We used nonlinear activation function softmax in the output layer, which converts inputs into a class of probabilities $\sigma(X)_j$, for intrusion detection classification. The output of the DNN is expressed as

$$\sigma(X)_j = \frac{e^{X_j}}{\sum_{k=1}^{k} e^{X_k}} \tag{12}$$

where X is a vector of inputs to the output layer and j indexes the number of output units, $j = 1, 2, \ldots, k$

With this DNN setup, the network training is carried out using the inputs to their respective class output (say for

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

normal 1 and abnormal 0). The DNN is trained with a vast training dataset, and the weight of each input connection is modified iteratively to minimize the errors occurring during training. The DNN has model parameters, and these parameters are tuned to train the network faster and better. These tuning parameters are known as hyperparameters which are used to control optimization functions and model selection during training with the learning algorithm. During the learning phase, these hyperparameters check whether the model under fits or overfits.

3.2.5 Optimization using Adam optimizer

The training is affected by the learning rate, which is one of the hyperparameters used in the DNN. It is necessary to choose efficient neural network architecture and the network parameters to minimize the errors. These have a direct effect on the neural network's performance. Adam optimizer is employed, which is the adaptive moment estimate to address this issue. This optimizer derives the learning rate using the first (mean) and second (variance) moments of the gradients [26]. The Adam algorithm is explained below and has resulted in better accuracy compared with other existing methods.

Algorithm Steps:

Input:

- Step size α
- Decay rates $\beta_1, \beta_2 \in (0,1)$
- Initial parameter θ

Initialization:

- 1st and 2nd moment estimates $a = 0, b = 0$
- Time step, $t = 0$

Adam update:

While early stopping criterion not met do

Sample a mini batch of D inputs from the training dataset, $X = [x_1, x_2, \ldots, x_D]$

Set $g = 0$

For $i = 1 to D$ do

Compute gradients: $g \leftarrow g + \nabla_\theta L(f(x_i;\theta), y_i)$

End for

$t \leftarrow t + 1$

Get biased 1st moment estimates: g $a \leftarrow \beta_1 a + (1-\beta_1) g$

Get biased 2nd moment estimates: $b \leftarrow \beta_2 b + (1-\beta_2) g$

Compute bias-corrected 1st moment estimate: $a' \leftarrow \frac{a}{1-\beta_1^t}$

Compute bias-corrected 2nd moment estimate: $b' \leftarrow \frac{b}{1-\beta_2^t}$

Compute update: $\Delta\theta = -\alpha \frac{a}{\sqrt{b} + \in} g\%$

Apply update: $\theta \leftarrow \theta - \Delta\theta$

End while

Return θ

3.3 Evaluation metrics

The following metrics are used to evaluate the performance of the proposed method.

Accuracy measures the proportion of the total number of correct classifications.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \tag{13}$$

Precision measures the number of correct classifications with the number of incorrect classifications.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{14}$$

Recall measures the number of correct classifications with the number of missed entries.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{15}$$

F1-score measures the harmonic mean of precision and recall.

$$\text{F1-score} = \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{16}$$

where TP—number of samples classified correctly as an attack class, FP—number of samples classified incorrectly as an attack class, TN—number of samples classified correctly as a normal class, FN—number of samples classified correctly as a normal class.

4 Results and discussion

This section discusses details of the hardware and software used in the experimentation of the proposed model. Our research work is based on the XGBoost–DNN, a supervised machine learning algorithm for intrusion detection. The deep learning method uses a deep neural network and softmax regression unit to classify network traffic as normal and attack. The proposed XGBoost–DNN classifier model is tested on the benchmark NSL-KDD dataset [27]. The experiments were performed using KERAS with the Tensor flow as background in the Anaconda3 environment [28] using python. The experiments were set up on Intel(R) Core(TM) i3-M380 CPU@ 2.53 GHz, 3.00 GB RAM, 64-bit Windows 7 OS.

The DNN is build with the reduced features as input nodes with three hidden layers and two output nodes for binary classification. The number of epochs given is 1000. During XGBoost-DNN training, the learning rate plays an important role in maintaining convergence. Unfortunately, we cannot analytically measure the optimal learning rate for a given model on a given dataset. Instead, a good

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

learning rate must be discovered via trial and error [29]. The difficulty in choosing a good learning rate is a prior reason; the adaptive learning rate methods are so useful and popular [30]. Adam, which is one of the popular adaptive learning rate methods, is used in our experimentation analysis. It converges very fast, and the learning speed of the model is quite fast and efficient. Also, it rectifies some of the problems faced by other optimization techniques such as vanishing learning rate, slow convergence, or high variance in the parameter updates, which causes loss function to fluctuate. For sparse datasets, adaptive learning rate works best than other optimization techniques like SGD, NAG, and momentum methods [31]. The learning rate is chosen among 8 learning rates such as 1E−0, 1E−1, 1E−2, 1E−3, 1E−4, 1E−5, 1E−6, and 1E−7. It is observed that the proposed XGBoost-DNN could learn the problem well with the learning rates 1E−1, 1E−2, and 1E−3, although successively giving rise to a slower convergence as the learning rate decreased. Figure 4 shows a plot with different learning rates and accuracies to demonstrate the relationship among them. With the chosen model configuration, the results suggest a moderate learning rate of 0.01, resulting in a good performance on the training and test sets.

For the better training of the proposed model, Adam optimizer is used with the learning rate of 0.01, respectively. The performance of the XGBoost–DNN classifier model is compared with the existing three shallow learning algorithms: logistic regression (LR), support vector machine (SVM), and naive Bayes (NB). All these comparisons are based on the same benchmark dataset NSL-KDD.

4.1 Performance evaluation

The proposed model is validated using *K*-fold cross-validation for intrusion detection. The NSL-KDD dataset is divided into the 5-, 10-, 15-, 20-, 25-folds for validation. K experiments, K − 1 fold, are used for training, and the remaining data are used for testing. Likewise, the process is repeated for K times, and the final result is taken as the cross-validation result. Deep learning deals with a high number of hidden layers and applies a higher-level learning algorithm such that it has stronger learning capability to obtain higher classification accuracy.

Also, the overall classification accuracy is different for different population sizes since deep neural networks (DNN) work better for problems with large population sizes [8]. Figures 5, 6, 7, 8, 9, 10, and 11 show the performance evaluation of the proposed model based on accuracy for different population sizes. The dataset is divided into different population sizes such as 1000, 2000, 3000, 4000, 5000, 6000, and 7000. It is carried out to

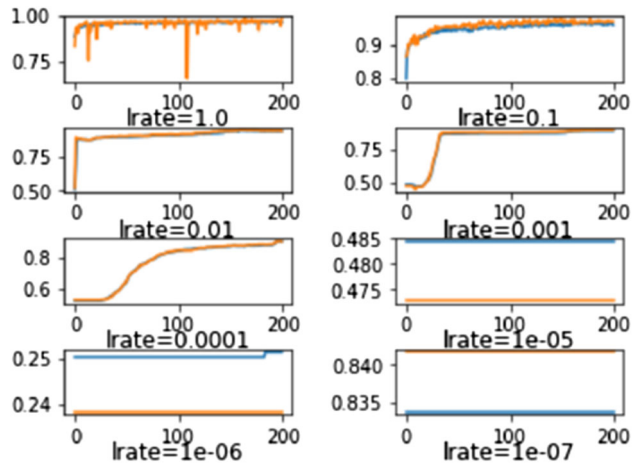

Fig. 4 Representation of relationship between different learning rates versus accuracies

analyze the accuracy for different population sizes using the proposed approach XGBoost–DNN classifier. From the experimental analysis, it is observed that the accuracy rate increases to the increase in the population size. With a population size of 7000 from Fig. 5, we have obtained the maximum accuracy of 97%. In Fig. 11, we observed the accuracy of 95% with a population size of 1000, which is less compared with the accuracy for a population size of 7000. Thus, the classification accuracy varies with respect to different population sizes. Hence, our proposed model demonstrates the characteristics of DNN.

4.2 Comparative analysis

This section discusses the comparative analysis of the proposed and the existing models with different *K*-folds. From Fig. 12, 10-fold cross-validation has shown a good accuracy compared with other *K*-fold validation. Tables 4, 5, 6, and 7 show the comparative analysis of the proposed model with other existing models. Logistic regression (LR), naive Bayes (NB), and support vector machine (SVM) are used for different population sizes. The results are based on the evaluation metrics such as accuracy, precision, recall, and F1-score. From the results, the accuracy, precision, recall, and F1-score are enhanced for the classification of network intrusions when compared with the other existing models. As per the analysis, the accuracy, precision, recall, and F1-score gradually increase as there is an increase in population size.

For the in-depth understanding of the proposed work, we also plotted receiver operating characteristic (ROC) shown in Fig. 13. Figure 13a shows the ROC curve for the proposed XGBoost–DNN model with a consistent accuracy with the AUC value of 0.98 with better true-positive and

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

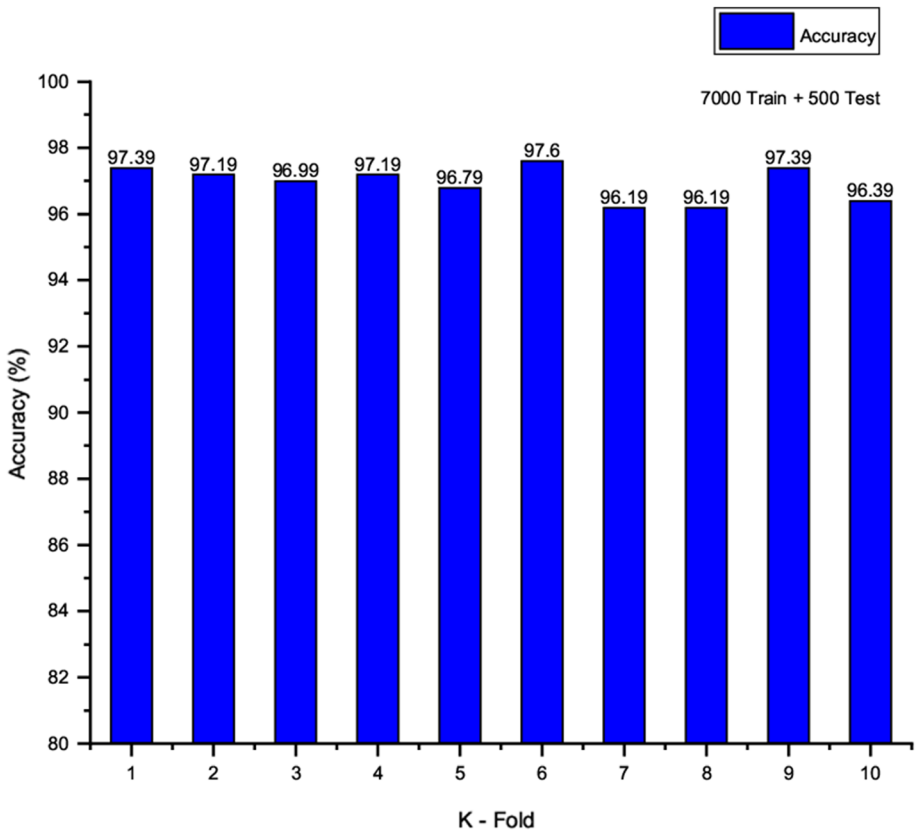

Fig. 5 Accuracy versus different *k*-folds for population size 7000

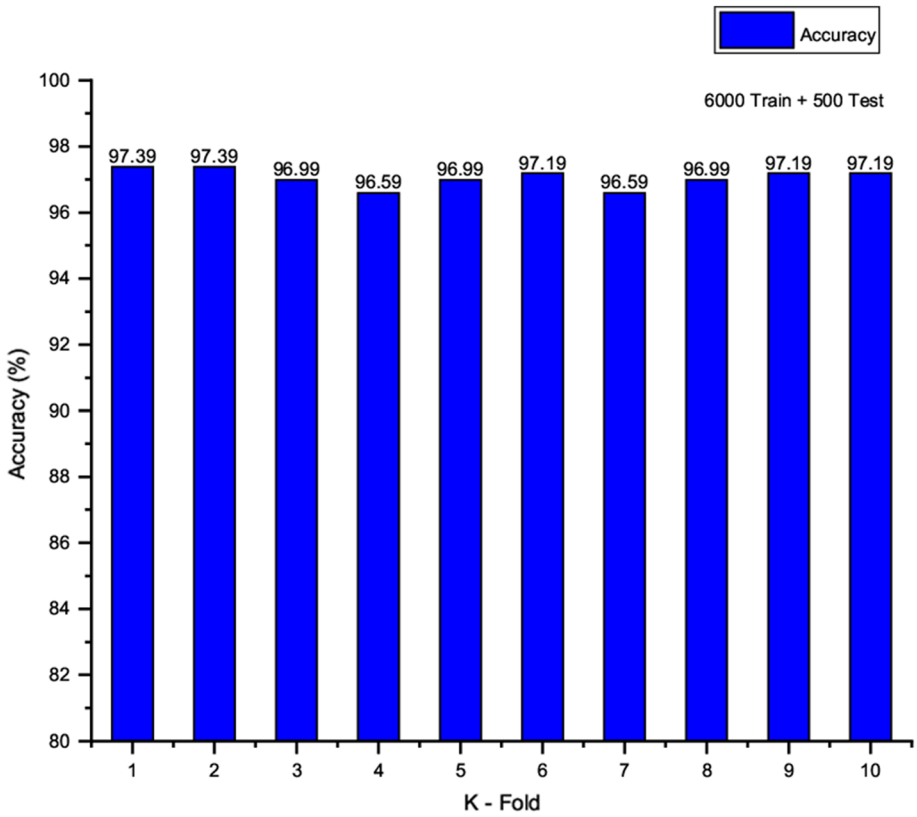

Fig. 6 Accuracy versus different *k*-folds for population size 6000

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

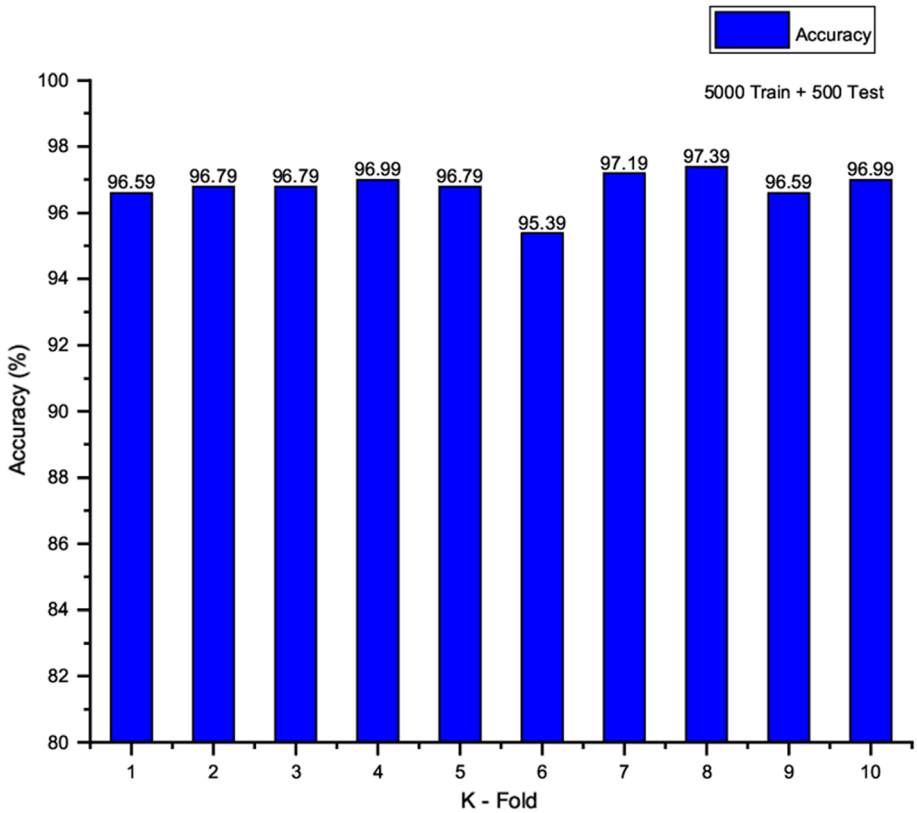

Fig. 7 Accuracy versus different *k*-folds for population size 5000

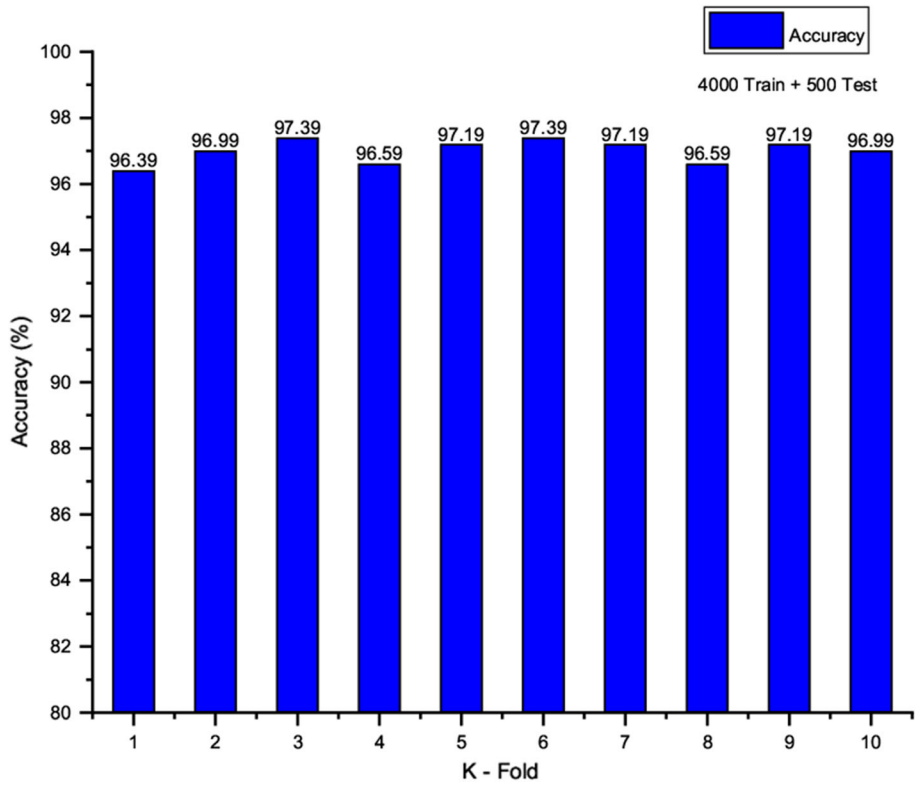

Fig. 8 Accuracy versus different *k*-folds for population size 4000

false-positive rates. Figure 13b shows the ROC curve for the existing machine learning approaches.

The experiments demonstrated that the proposed XGBoost-DNN significantly outperformed the existing

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

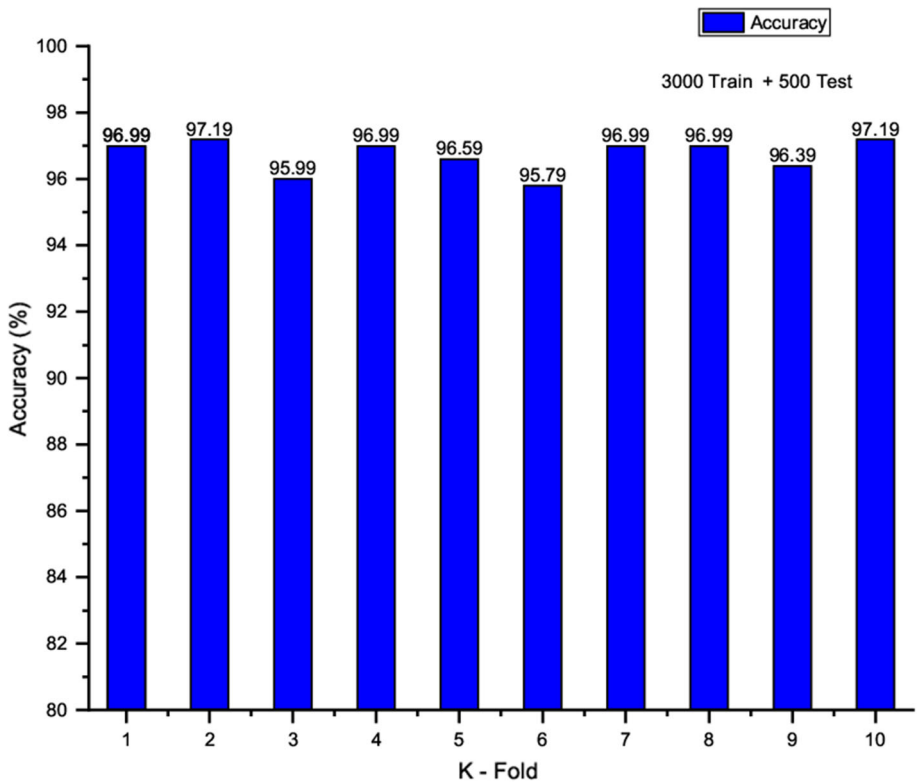

Fig. 9 Accuracy versus different *k*-folds for population size 3000

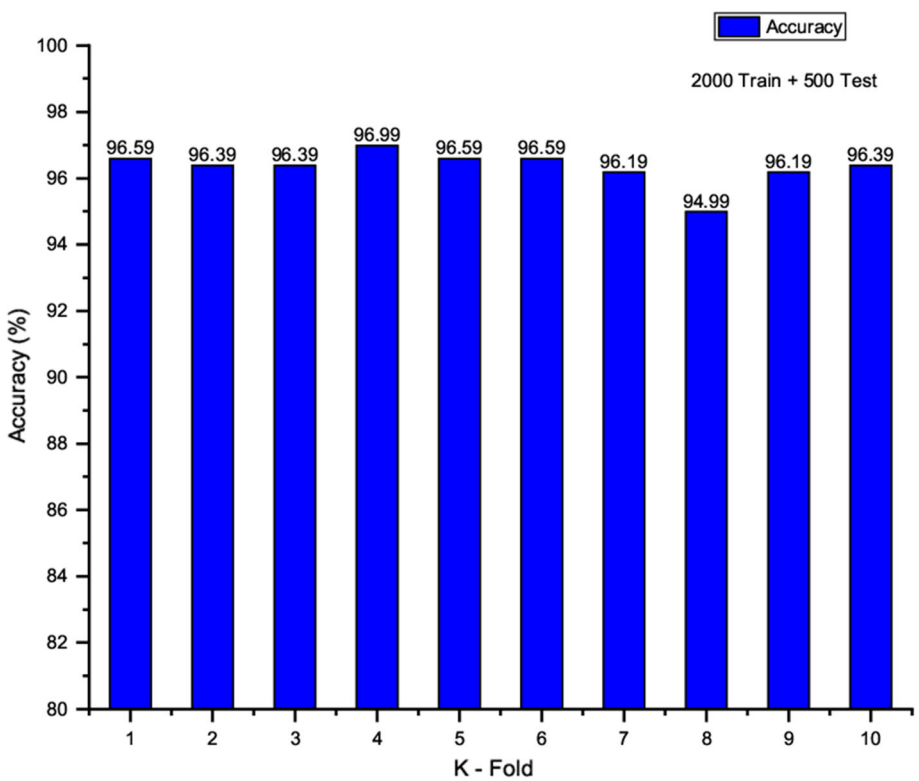

Fig. 10 Accuracy versus different *k*-folds for population size 2000

shallow network in the detection of attacks. The deep autoencoder, when trained with backpropagation, has not been a success; the evaluation gets stuck in local minima with a minimal gradient signal when trained with backpropagation [32]. Also, issues such as classification of data, high-level preprocessing, suppression, and mitigation of

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

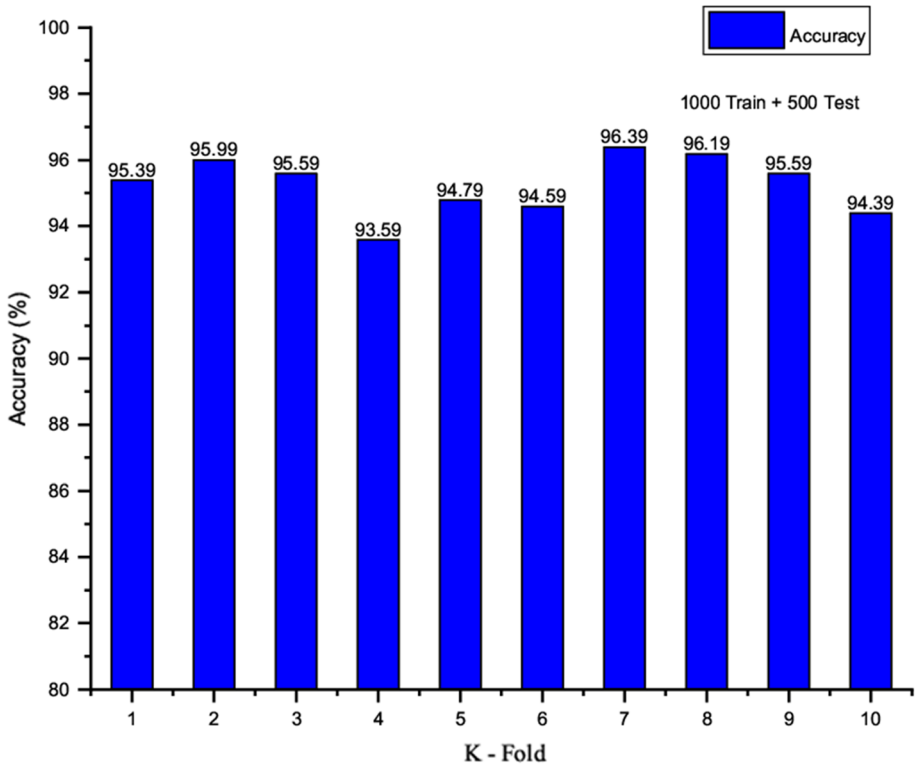

Fig. 11 Accuracy versus different *k*-folds for population size 1000

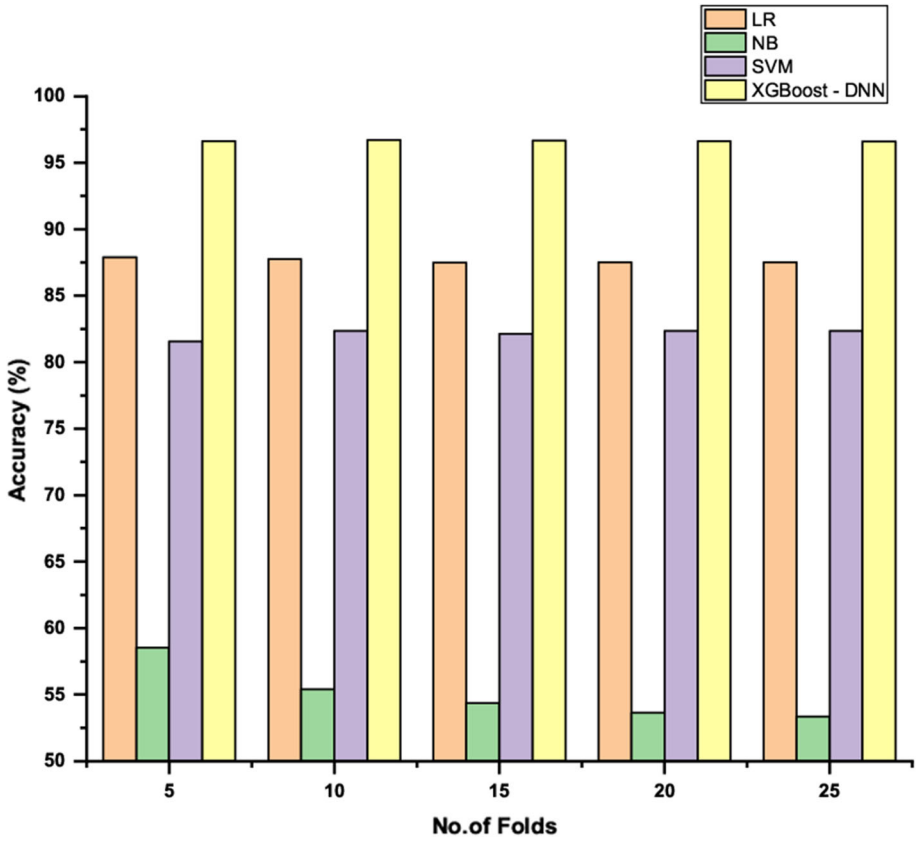

Fig. 12 Comparative analysis of the proposed model and the existing models with different *K*-folds

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 4 Comparative analysis of the proposed model and the existing models on the basis of accuracy

Population size	Accuracy			
	XGBoost–DNN	LR	NB	SVM
1000	0.953	0.91	0.85	0.73
2000	0.963	0.88	0.83	0.78
3000	0.967	0.88	0.55	0.82
4000	0.969	0.87	0.53	0.88
5000	0.971	0.86	0.52	0.89
6000	0.974	0.87	0.52	0.89
7000	0.976	0.87	0.52	0.90

Table 5 Comparative analysis of the proposed model and the existing models on the basis of precision

Population size	Precision			
	XGBoost–DNN	LR	NB	SVM
1000	0.94	0.91	0.85	0.81
2000	0.96	0.92	0.83	0.83
3000	0.96	0.87	0.29	0.86
4000	0.96	0.87	0.38	0.88
5000	0.97	0.87	0.27	0.89
6000	0.97	0.87	0.27	0.89
7000	0.97	0.87	0.28	0.90

Table 6 Comparative analysis of the proposed model and the existing models on the basis of recall

Population size	Recall			
	XGBoost–DNN	LR	NB	SVM
1000	0.94	0.91	0.85	0.70
2000	0.96	0.92	0.83	0.75
3000	0.96	0.87	0.54	0.80
4000	0.96	0.87	0.53	0.89
5000	0.97	0.87	0.52	0.89
6000	0.97	0.87	0.52	0.89
7000	0.97	0.87	0.52	0.90

DDoS attacks, false alarm rates, and the semi-supervised approaches need to be dealt with, to come up with a robust IDS model [23] and this paper is a step toward achieving it. Most of the classification methods discussed above had their advantages and limitations; XGBoost was chosen because it helped in tackling the majority of the flaws that emerged from the existing models. The reasons XGBoost is handpicked to be the preferred classification model when dealing with issues encountered in real-word classification problems are the following [23]:

Table 7 Comparative analysis of the proposed model and the existing models on the basis of F1-score

Population size	F1-score			
	XGBoost–DNN	LR	NB	SVM
1000	0.97	0.87	0.85	0.66
2000	0.96	0.92	0.83	0.73
3000	0.96	0.91	0.38	0.79
4000	0.96	0.87	0.37	0.88
5000	0.97	0.87	0.36	0.89
6000	0.97	0.87	0.36	0.89
7000	0.97	0.87	0.36	0.90

- XGBoost is approximately 10 times faster than existing methods on a single platform.
- XGBoost has the advantage of parallel processing, which uses all the cores of the machine it is running on.
- The portability of XGBoost makes it available and more comfortable to blend on many platforms. In recent days, the distributed versions have been integrated into cloud platforms such as Tianchi of Alibaba, AWS, GCE, and Azure. Therefore, the flexibility offered by XGBoost is immense and not tied to a specific platform; hence, the IDS using XGBoost can be platform independent, which is a significant advantage.
- Regularization is an essential aspect of XGBoost algorithm, as it helps in avoiding data overfitting problems whether it is tree-based or linear models.

All these essential functionalities add up and enable the XGBoost to outperform many existing models. Moreover, XGBoost has been used in many award winning solutions in machine learning competitions conducted by Kaggle (17 out of 29 winning solutions in 2015) and KDD Cup (top 10 winning solutions all used XGBoost in 2015) [33]. Therefore, the proposed XGBoost-DNN is very well equipped to deal with the majority of the problems that face a real-world network.

5 Conclusion

In this paper, we have discussed the shortcomings in existing IDS and also an earnest attempt has been made to implement XGBoost–DNN-based classification model for the intrusion detection system. XGBoost deals with regularization and helps in preventing overfitting issues. XGBoost provides faster detection of network intrusions than the existing models applied for binary classification of intrusion detection. XGBoost model is primarily used for classification, and in our work, we have utilized the feature importance score generated by XGBoost model for feature

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

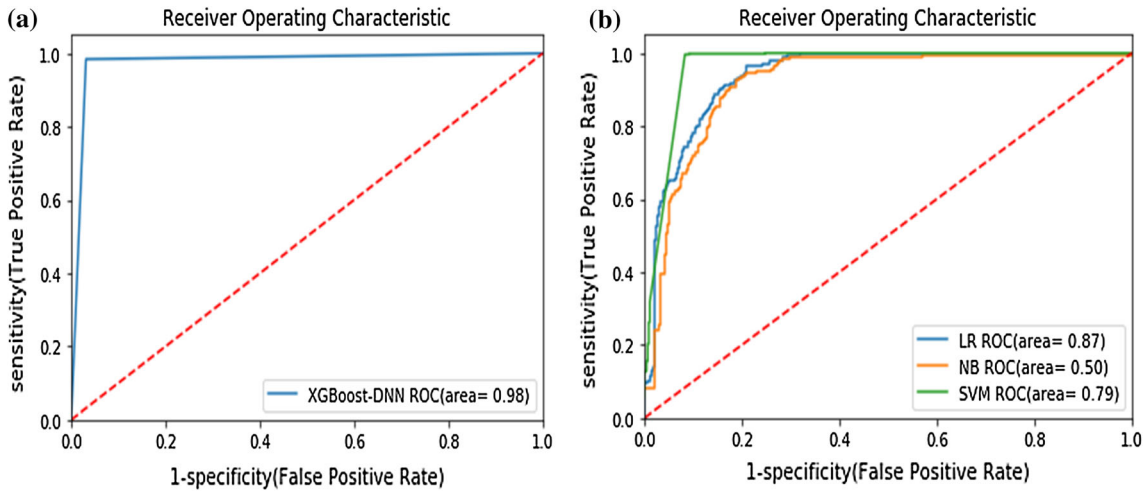

Fig. 13 ROC curve for **a** existing models, **b** proposed model

selection. This feature selection method is first of its kind, which is applied for intrusion detection and DNN for classification of intrusions using NSL-KDD dataset. The results of the same are compared with existing logistic regression, naive Bayes, and support vector machine. The efficient classifier was identified based on the results of accuracy, precision, recall, and F1-score as well as ROC curve. From the observed results, a deep learning model reveals a consistent level of 97% classification accuracy than existing models. It has the edge over others. The proposed work is applied for binary classification. In the near future, the work can be extrapolated and stretched for multiclass classification.

References

1. Shone N, Ngoc TN, Phai VD, Shi Q (2018) A deep learning approach to network intrusion detection. IEEE Trans Emerg Top Comput Intell 2:41–50
2. Mabu S, Chen C, Lu N, Shimada K (2011) An Intrusion-detection model based on fuzzy programming. IEEE Trans Syst Man Cybern Syst 41:130–139
3. Garcia-Teodoro P, Diaz-Verdejo J, Macia-Fernandez G, Vázquez E (2009) Anomaly-based network Intrusion detection: techniques, systems and challenges. Comput Secur 28:18–28
4. Li L, Yu Y, Bai S, Hou Y, Chen X (2017) An effective two-step intrusion detection approach based on binary classification and k-NN. IEEE Access 6:12060–12073
5. Ahmad I, Basheri M, Iqbal MJ, Rahim A (2018) Performance comparison of support vector machine, random forest, and extreme learning machine for intrusion detection. IEEE Access 6:33789–33795
6. Farnaaz N, Jabbar MA (2016) Random forest modeling for network intrusion detection system. Procedia Comput Sci 89:213–217
7. Manzoor I, Kumar N (2017) A feature reduced intrusion detection system using ANN classifier. Expert Syst Appl 88:249–257
8. LeCun Y, Bengio Y, Hinton G (2015) Deep learning. Nature 521:436–444
9. Chen T, Guestrin C (2016) XGBoost: a scalable tree boosting system. In: the proceedings of 22nd SIGKDD conference on knowledge discovery and data mining. ACM, New York
10. Javaid A, Niyaz Q, Sun W, Alam M (2016) A deep learning approach for network intrusion detection system. In: The proceedings of the 9th EAI international conference on bio-inspired information and communications technologies
11. Kim J, Shin N, Jo SY, Kim SH (2017) Method of intrusion detection using deep neural network. In: IEEE international conference on big data and smart computing (BigComp), IEEE
12. Paul S, Banerjee C, Ghoshal M (2018) A CFS–DNN-based intrusion detection system. In: Bera R, Sarkar S, Chakraborty S (eds) Advances in communication, devices and networking. Springer, Berlin
13. Zhang C, Wang X, Li F, He Q, Huang M (2018) Deep learning-based network application classification for SDN. T Emerg Telecommun Technol. https://doi.org/10.1002/ett.3302
14. Aljawarneh S, Aldwairi M, Yassein MB (2018) Anomaly-based intrusion detection system through feature selection analysis and building hybrid efficient model. J Comput Sci 25:152–160
15. Moustafa N, Turnbull B, Choo KK (2018) An ensemble intrusion detection technique based on proposed statistical flow features for protecting network traffic of internet of things. IEEE Internet Things J 6(3):4815–4830
16. Tavallaee M, Bagheri E, Lu W, Ghorbani AA (2009) A detailed analysis of the KDD CUP 99 data set. In: IEEE symposium on computational intelligence for security and defense applications, IEEE
17. Vinayakumar R, Alazab M, Soman KP, Poornachandran P, Al-Nemrat A, Venkatraman S (2019) Deep learning approach for intelligent intrusion detection system. IEEE Access 7:41525–41550
18. Wang Z (2018) Deep learning-based intrusion detection with adversaries. IEEE Access 6:38367–38384
19. Behera S, Pradhan A, Dash R (2018) Deep neural network architecture for anomaly based intrusion detection system. In: 5th international conference on signal processing and integrated networks (SPIN), IEEE
20. Mishra P, Varadharajan V, Tupakula U, Pilli ES (2018) A detailed investigation and analysis of using machine learning

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

techniques for intrusion detection. IEEE Commun Surv Tuts 21(1):686–728
21. Priddy KL, Keller PE (2005) Artificial Neural Networks: An introduction. SPIE Press, Bellingham. https://doi.org/10.1117/3.633187
22. Li S, Zhang X (2019) Research on orthopedic auxiliary classification and prediction model based on XGBoost algorithm. Neural Comput App. https://doi.org/10.1007/s00521-019-04378-4
23. Dhaliwal S, Nahid AA, Abbas R (2018) Effective intrusion detection system using XGBoost. Information 9(7):149
24. Chen Z, Jiang F, Cheng Y, Gu X, Liu W, Peng J (2018) XGBoost classifier for DDoS attack detection and analysis in SDN-Based cloud. In: IEEE international conference on big data and smart computing (BigComp), IEEE
25. Glorot X, Bordes A, Bengio Y (2011) Deep sparse rectifier neural networks. In: Proceedings of 14th international conference on artificial intelligence and statistics, IEEE
26. Kingma DP, Ba J (2014) Adam: a method for stochastic optimization. arXiv preprint arXiv:1412.6980
27. NSL-KDD Dataset (2018) Publicly available dataset. https://www.unb.ca/cic/datasets/nsl.html. Accessed 9 Apr 2018
28. Anaconda Software Distribution (2018). Computer software. Vers. 3. Anaconda. https://anaconda.com. Accessed 22 Apr 2018
29. Brownlee J (2019) How to configure the learning rate when training deep learning neural networks. https://machinelearningmastery.com/learning-rate-for-deep-learning-neural-networks/. Accessed 20 June 2019
30. Goodfellow I, Bengio Y, Courville A (2016) Optimization for training deep models. In: Deep learning. MIT Press, Cambridge. http://www.deeplearningbook.org
31. Walia AS (2019) Types of optimization algorithms used in neural networks and ways to optimize gradient descent. https://towardsdatascience.com/types-of-optimization-algorithms-used-in-neural-networks-and-ways-to-optimize-gradient 95ae5d39529f. Accessed 20 June 2019
32. Hodo E, Bellekens X, Hamilton A, Tachtatzis C, Atkinson R (2017) Shallow and deep networks intrusion detection system: a taxonomy and survey. arXiv preprint arXiv:1701.02145
33. Chen T, Guestrin C (2016) XGBoost: a scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining. ACM, New York

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH ("Springer Nature").

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users ("Users"), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use ("Terms"). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval , sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com